

Vue Prop 校验、默认值与循环遍历的使用及注意事项

一、Prop 校验与默认值（Vue 2/3 通用）

Prop 是 Vue 组件间通信的核心方式，**Prop 校验** 用于规范传入的属性类型、格式，**默认值** 则为属性提供兜底值，避免未传值时出现异常。

1. 基础语法（Vue 3 组合式 API + 选项式 API 双版本）

(1) Vue 3 选项式 API（与 Vue 2 一致）

Code block

```
1  <template>
2    <div class="user-card">
3      <h3>{{ user.name }}</h3>
4      <p>年龄: {{ user.age }}</p>
5      <p>标签: {{ tags.join(', ') }}</p>
6    </div>
7  </template>
8
9  <script>
10 export default {
11   // 定义 props 并配置校验和默认值
12   props: {
13     // 基础类型校验 + 默认值
14     title: {
15       type: String, // 类型: String/Number/Boolean/Array/Object/Function/Symbol
16       required: false, // 是否必传
17       default: '用户信息卡片', // 基础类型默认值直接写
18       validator: (value) => {
19         // 自定义校验规则: 标题长度不超过 20
20         return value.length <= 20;
21       }
22     },
23
24     // 对象类型校验 + 默认值（必须是函数返回，避免引用类型共享）
25     user: {
26       type: Object,
27       required: true, // 必传属性
28       default: () => ({ // 对象/数组默认值必须是工厂函数
29         name: '未知用户',
```

```

30         age: 0
31     }),
32     validator: (value) => {
33         // 自定义校验：必须包含 name 和 age 属性
34         return 'name' in value && 'age' in value;
35     }
36 },
37
38 // 数组类型校验 + 默认值
39 tags: {
40     type: Array,
41     default: () => ['默认标签'], // 数组默认值也需工厂函数
42 },
43
44 // 多类型校验
45 status: {
46     type: [String, Number], // 可以是字符串或数字
47     default: 'normal'
48 },
49
50 // 布尔类型（注意：传入时 <component :is-show="false" /> 才是布尔值，直接写 is-
show 会被解析为 true）
51 isShow: {
52     type: Boolean,
53     default: true
54 }
55 },
56 mounted() {
57     console.log(this.title, this.user, this.tags);
58 }
59 };
60 </script>

```

(2) Vue 3 组合式 API (`<script setup>`)

Code block

```

1  <template>
2    <div class="user-card">
3      <h3>{{ title }}</h3>
4      <p>{{ user.name }} ({{ user.age }}岁)</p>
5      <p>标签: {{ tags.join(', ') }}</p>
6    </div>
7  </template>
8
9  <script setup>
10 import { defineProps, withDefaults } from 'vue';

```

```

11
12 // 方式1: 基础定义 (仅类型, 无默认值/校验)
13 // const props = defineProps(['title', 'user', 'tags']);
14
15 // 方式2: 完整定义 (类型 + 默认值 + 校验)
16 const props = withDefaults(
17   defineProps({
18     title: {
19       type: String,
20       validator: (v) => v.length <= 20
21     },
22     user: {
23       type: Object,
24       required: true,
25       validator: (v) => 'name' in v && 'age' in v
26     },
27     tags: Array,
28     status: [String, Number],
29     isShow: Boolean
30   }),
31   {
32     // 配置默认值 (对象/数组无需工厂函数, Vue 3 自动处理)
33     title: '用户信息卡片',
34     tags: () => ['默认标签'], // 数组仍建议用工厂函数 (避免跨实例共享)
35     status: 'normal',
36     isShow: true
37   }
38 );
39
40 // 使用 props
41 console.log(props.title, props.user);
42 </script>

```

2. 关键注意事项

- **默认值类型限制:**
 - 基础类型 (String/Number/Boolean) : 直接写值 `default: 'xxx'`
 - 引用类型 (Object/Array) : 必须用工厂函数 `default: () => ({})` (Vue 2) , Vue 3 组合式 API 可简化, 但仍建议用工厂函数避免共享引用
- **required 优先级:** `required: true` 时, 默认值会失效 (必传属性无需默认值)
- **自定义校验:** `validator` 函数返回布尔值, 返回 `false` 时控制台会警告
- **布尔类型传值:** 模板中直接写 `<my-component is-show />` 会被解析为 `true` , 传 `false` 需用绑定 `<my-component :is-show="false" />`

二、循环遍历数组/对象（v-for）

`v-for` 是 Vue 渲染列表的核心指令，支持遍历数组、对象、数字、字符串，需配合 `key` 保证列表更新性能。

1. 遍历数组（最常用）

(1) 基础遍历（值 + 索引）

Code block

```
1  <template>
2    <ul>
3      <!-- 遍历数组: item 是当前项, index 是索引 (可选) -->
4      <li v-for="(item, index) in userList" :key="index">
5        {{ index + 1 }}. {{ item.name }} - {{ item.age }}岁
6      </li>
7    </ul>
8  </template>
9
10 <script setup>
11 import { ref } from 'vue';
12
13 // 定义数组对象
14 const userList = ref([
15   { id: 1, name: '张三', age: 20 },
16   { id: 2, name: '李四', age: 22 },
17   { id: 3, name: '王五', age: 25 }
18 ]);
19 </script>
```

(2) key 的最佳实践

- 优先用**唯一标识**（如 id）作为 key，而非索引（索引会随数组增删变化，导致 DOM 复用异常）：

Code block

```
1  <li v-for="item in userList" :key="item.id">
2    {{ item.name }} - {{ item.age }}岁
3  </li>
```

2. 遍历对象（值 + 键 + 索引）

Code block

```
1  <template>
2    <div class="user-info">
```

```

3      <!-- 遍历对象: value 是值, key 是键, index 是索引 (可选) -->
4      <p v-for="(value, key, index) in userInfo" :key="key">
5          {{ index + 1 }}. {{ key }}: {{ value }}
6      </p>
7  </div>
8 </template>
9
10 <script setup>
11 import { ref } from 'vue';
12
13 const userInfo = ref({
14     name: '张三',
15     age: 20,
16     gender: '男',
17     address: '北京市'
18 });
19 </script>

```

3. 嵌套遍历 (数组对象嵌套)

Code block

```

1  <template>
2    <div class="group-list">
3      <!-- 外层遍历分组 -->
4      <div v-for="group in groupList" :key="group.id" class="group">
5        <h3>{{ group.groupName }}</h3>
6        <!-- 内层遍历分组下的用户 -->
7        <ul>
8          <li v-for="user in group.users" :key="user.id">
9            {{ user.name }} ({{ user.age }}岁)
10          </li>
11        </ul>
12      </div>
13    </div>
14  </template>
15
16 <script setup>
17 import { ref } from 'vue';
18
19 const groupList = ref([
20     {
21       id: 1,
22       groupName: '研发部',
23       users: [{ id: 101, name: '张三', age: 28 }, { id: 102, name: '李四', age:
30   }]

```

```

24     },
25     {
26       id: 2,
27       groupName: '产品部',
28       users: [{ id: 201, name: '王五', age: 26 }, { id: 202, name: '赵六', age:
29       29 }]
30     }
31   ]});
32 </script>

```

4. 遍历注意事项

- **key 必须唯一**：同一父节点下的 `v-for` 项 key 不能重复，否则控制台警告
- **避免 v-for 与 v-if 同节点**：`v-for` 优先级高于 `v-if`，会导致性能浪费，建议：

Code block

```

1  <!-- 推荐：先过滤数组，再遍历 -->
2  <li v-for="item in filteredList" :key="item.id">...</li>
3
4  <script setup>
5    const filteredList = computed(() => userList.value.filter(item => item.age
6    > 20));
7  </script>

```

- **响应式更新**：Vue 2 中修改数组需用变异方法（push/splice/shift 等）或替换数组；Vue 3 中直接修改数组索引（`userList.value[0] = {}`）也能响应式更新（基于 Proxy）

三、综合示例（Prop + 循环遍历）

Code block

```

1  <!-- 子组件 UserList.vue -->
2  <template>
3    <div class="user-list">
4      <h2>{{ title }}</h2>
5      <div v-if="users.length === 0" class="empty">暂无用户数据</div>
6      <ul v-else>
7        <li v-for="user in users" :key="user.id" class="user-item">
8          <span>{{ user.name }}</span>
9          <span>{{ user.age }}岁</span>
10         <span v-for="tag in user.tags" :key="tag" class="tag">{{ tag }}</span>
11       </li>
12     </ul>
13   </div>

```

```
14 </template>
15
16 <script setup>
17 import { defineProps, withDefaults } from 'vue';
18
19 // 定义 props 校验和默认值
20 const props = withDefaults(
21   defineProps({
22     title: {
23       type: String,
24       validator: (v) => v.length <= 30
25     },
26     users: {
27       type: Array,
28       required: true,
29       validator: (list) => {
30         // 校验每个用户对象包含 id/name/age 属性
31         return list.every(item => 'id' in item && 'name' in item && 'age' in
item);
32       }
33     }
34   }),
35   {
36     title: '用户列表'
37   }
38 );
39 </script>
40
41 <!-- 父组件 App.vue -->
42 <template>
43   <UserList
44     title="研发部用户列表"
45     :users="userData"
46   />
47 </template>
48
49 <script setup>
50 import UserList from './components/UserList.vue';
51 import { ref } from 'vue';
52
53 const userData = ref([
54   { id: 1, name: '张三', age: 28, tags: ['前端', 'Vue'] },
55   { id: 2, name: '李四', age: 30, tags: ['后端', 'Java'] },
56   { id: 3, name: '王五', age: 26, tags: ['测试', '自动化'] }
57 ]);
58 </script>
```

总结

1. **Prop 校验**: 规范传入属性, 提升组件健壮性, 核心配置:

`type` / `required` / `default` / `validator`

2. **默认值**: 基础类型直接赋值, 引用类型用工厂函数 (Vue 2), Vue 3 可简化但建议保持一致

3. **v-for 遍历**: 数组遍历用 `(item, index)`, 对象遍历用 `(value, key, index)`, 必须配合唯一 `key`

4. **性能与规范**: 避免 `v-for` 与 `v-if` 同节点, 优先用唯一标识作为 `key`, 保证列表更新效率

(注: 文档部分内容可能由 AI 生成)